

Proyecto Final: Sistema de Matrícula Universitaria por Consola

Estudiante: Miguel Angel Peralta Cano

Programa: Ingeniería de software

Materia: Lógica de programación 1

Unidad: 3

Docente: JOHAN MANUEL GORDILLO MESA

31/05/2026

Contents

1	Introducción	3
2	Estructuras de Datos Utilizadas	3
2.1	Definición del Catálogo	3
2.2	Registro del Estudiante	3
3	Preguntas Frecuentes y Validaciones	3
3.1	¿Por qué se usan varias listas en paralelo en lugar de una sola lista de objetos?	3
3.2	¿Qué pasa si intentas inscribir más cupos de los disponibles en el curso? . .	4
3.3	¿En qué parte del código se usan los operadores <code>&&</code> , <code>>=</code> y <code>+=</code> ?	4
4	Instrucciones de Compilación	5
5	Conclusiones	5

1 Introducción

El presente proyecto consiste en el desarrollo de un sistema de matrícula para la Universidad SALLE, implementado en Java. El sistema permite gestionar un catálogo de cursos, realizar inscripciones de estudiantes, aplicar descuentos y calcular el total de la matrícula bajo condiciones específicas de beca, aplicando los conceptos vistos en clase.

2 Estructuras de Datos Utilizadas

En lugar de emplear Programación Orientada a Objetos (la cual se aborda en cursos posteriores), el sistema hace uso de **Listas Paralelas** implementadas con `ArrayList`.

2.1 Definición del Catálogo

Se utilizan tres listas globales en paralelo para mantener la información de los cursos. El índice i en la lista `nombresCursos` se corresponde exactamente con el índice i en `valoresCursos` y `cuposCursos`.

```
1 static ArrayList<String> nombresCursos = new ArrayList<>(Arrays.asList(  
2     "Logica de programacion", "Estructuras discretas", ...));  
3  
4 static ArrayList<Double> valoresCursos = new ArrayList<>(Arrays.asList(  
5     180.0, 200.0, ...));  
6  
7 static ArrayList<Integer> cuposCursos = new ArrayList<>(Arrays.asList(  
8     35, 30, ...));
```

2.2 Registro del Estudiante

Para el estudiante actual de la sesión, se mantienen dos listas paralelas que almacenan sus elecciones:

```
1 static ArrayList<String> matriculaCursos = new ArrayList<>();  
2 static ArrayList<Double> matriculaValores = new ArrayList<>();
```

Estas listas se vacían mediante `.clear()` cuando se efectúa un cambio de usuario, preservando así el estado del catálogo global.

3 Preguntas Frecuentes y Validaciones

3.1 ¿Por qué se usan varias listas en paralelo en lugar de una sola lista de objetos?

En el contexto de introducción a la lógica de programación, el uso de listas en paralelo sirve como una excelente herramienta didáctica para comprender la gestión de índices, arreglos dinámicos (`ArrayList`) y sincronización de datos mediante estructuras iterativas (como los bucles `for`). Al no utilizar una clase `Curso` (Programación Orientada a Objetos), se refuerza la destreza del desarrollador para manipular tipos primitivos y sincronizar estados lógicamente a través de una misma posición en memoria indexada.

3.2 ¿Qué pasa si intentas inscribir más cupos de los disponibles en el curso?

El sistema previene que los cursos queden sobrevendidos o con cupos negativos. En la función `inscribirCurso()`, se evalúa la disponibilidad actual del curso solicitado:

```
1 if (cuposCursos.get(id) > 0) {
2     matriculaCursos.add(cursoNombre);
3     matriculaValores.add(valoresCursos.get(id));
4     cuposCursos.set(id, cuposCursos.get(id) - 1);
5     System.out.println("Listo! Te has inscrito en " + cursoNombre);
6 } else {
7     System.out.println("Lo sentimos, no quedan cupos para este curso.");
8 }
```

Si la condición `cuposCursos.get(id) > 0` no se cumple (es decir, los cupos han llegado a 0), el sistema bloquea la inscripción, muestra un mensaje informando la falta de disponibilidad y no realiza ningún cobro ni registro en la matrícula del estudiante. Además, cada estudiante está restringido a inscribir un curso específico una sola vez por semestre, verificado mediante `matriculaCursos.contains(cursoNombre)`.

3.3 ¿En qué parte del código se usan los operadores `&&`, `>=` y `+=`?

Estos operadores son fundamentales para las validaciones y cálculos matemáticos dentro del sistema:

- **Operador Lógico AND (`&&`):** Se utiliza para verificar que múltiples condiciones se cumplan al mismo tiempo.
 - En `inscribirCurso()`: `if (id >= 0 && id < nombresCursos.size())` garantiza que el índice ingresado por el usuario no cause un `IndexOutOfBoundsException`.
 - En `agregarCursoCatalogo()`: `if (!nombre.isEmpty() && valor > 0 && cupos >= 0)` asegura que un administrador no ingrese cursos con precios negativos o sin nombre.
- **Operador Relacional Mayor o Igual (`>=`):** Se usa para validar topes y rangos.
 - En `inscribirCurso()`: `if (matriculaCursos.size() >= MAX_CURSOS_ESTUDIANTE)` limita a los alumnos a 5 materias.
 - En `calcularBecaAutomatica()`: `if (total >= 1000), else if (total >= 600) y else if (total >= 200)` determinan la categorización de la beca del estudiante en base a su subtotal de matrícula.
- **Operador de Asignación y Suma Acumulativa (`+=`):** Se emplea para acumular valores en una misma variable durante un bucle.
 - En `verMatricula()`: `total += matriculaValores.get(i);` va sumando el costo de cada materia al total global.
 - En `calcularBecaAutomatica()`: `for (double v : matriculaValores) total += v;` calcula rápidamente el monto bruto para determinar si el estudiante aplica a algún nivel de beca.

4 Instrucciones de Compilación

Para ejecutar el programa, asegúrese de tener instalado el JDK y siga estos pasos:

```
1 # Compilar
2 javac src/App.java -d bin/
3
4 # Ejecutar
5 java -cp bin/ App
```

5 Conclusiones

El sistema de consola actual cumple con creces los requerimientos de manejo de colecciones, flujo de ejecución y lógica relacional, asegurando la integridad transaccional (cupos globales consistentes entre múltiples estudiantes). La implementación actual representa un diseño procedimental óptimo con base a los requerimientos de la asignatura de lógica de programación.